



INSTITUTO SUPERIOR TECNOLÓGICO RUMIÑAHUI

MANUAL DE ADMINISTRADOR

TaskMaster Web — Sistema de Gestión de Productividad Personal

Versión 1.0 — Agosto 2026

Eduardo Antonio Hurtado Quintero

Ing: Llumiquinga Espinoza Fabricio Jose

Sangolquí – Ecuador, Agosto 2026

1. Introducción

Este manual está dirigido al administrador del sistema TaskMaster Web. Cubre la instalación, configuración, gestión de la base de datos, monitoreo del servidor, administración de usuarios, configuración de integraciones y procedimientos de mantenimiento.

El administrador es responsable de garantizar la disponibilidad, seguridad y correcto funcionamiento del sistema para todos los usuarios.

1.1 Perfil del administrador

- Conocimientos básicos de Node.js y npm
- Manejo de Firebase Console y Firestore
- Acceso al servidor de hosting (Hostinger o Railway)
- Conocimientos básicos de MySQL y phpMyAdmin
- Acceso al repositorio GitHub del proyecto

2. Instalación del Sistema en Entorno Local

2.1 Requisitos del servidor

Componente	Versión mínima requerida
Node.js	18 LTS o superior (recomendado: 20 LTS)
npm	9.0 o superior (incluido con Node.js)
MySQL	8.0 o superior (incluido en XAMPP)
XAMPP	8.2+ con Apache en puerto 800 y MySQL en 3306
Git	2.40 o superior
Cuenta Firebase	Proyecto activo en console.firebase.google.com

2.2 Clonar el repositorio

```
git clone https://github.com/valeriafarinango0-cyber/TaskMaster-Web.git
cd TaskMaster-Web
git checkout avance-eduardo
```

2.3 Instalar dependencias

```
# Servidor principal
npm install
```

```
# Servidor de integraciones
cd integraciones
npm install
cd ..
```

2.4 Configurar variables de entorno

Copia el archivo `.env.example` a `.env` y configura los valores:

```
cp .env.example .env
```

Contenido del archivo `.env`:

```
PORT=3000
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
DB_NAME=taskmaster_db
GOOGLE_CLIENT_ID=tu-client-id.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=tu-client-secret
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=taskmaster@gmail.com
EMAIL_PASS=tu-app-password
```

 *Nunca subir el archivo `.env` a GitHub. Está incluido en `.gitignore` para que no se suba automáticamente.*

2.5 Configurar Firebase Admin SDK

El archivo de credenciales de Firebase Admin ya está en la raíz del proyecto (`taskmaster-web-dbcd0-firebase-adminsdk-fbsvc-7ae8ae36dd.json`). Verifica que `lib/firestore.js` apunte a ese archivo.

 *Este archivo contiene credenciales privadas. No lo compartas ni lo subas a repositorios públicos.*

2.6 Crear la base de datos MySQL

Paso 1: Abrir XAMPP

Inicia Apache y MySQL desde el panel de control de XAMPP.

Paso 2: Abrir phpMyAdmin

Ve a <http://127.0.0.1:800/phpmyadmin> en el navegador.

Paso 3: Crear BD

Clic en "Nueva" → nombre: taskmaster_db → Crear.

Paso 4: Importar SQL

Selecciona taskmaster_db → pestaña Importar → sube database/taskmaster_db.sql → Continuar.

2.7 Iniciar el servidor

```
npm start
# Salida esperada:
# TaskMaster Node server running on http://localhost:3000
#  Firebase Firestore conectado correctamente
```

 El servidor de integraciones se inicia por separado: `cd integraciones && npm start`

3. Gestión de Usuarios

3.1 Ver usuarios registrados

Los usuarios se gestionan desde dos fuentes: Firebase Firestore (usuarios que usan la app web) y MySQL (respaldo local).

En Firebase Console:

Paso 1: Abrir Firebase Console

Ve a console.firebase.google.com → selecciona el proyecto taskmaster-web.

Paso 2: Ver usuarios

Ve a Authentication → Users. Aquí están todos los usuarios registrados con su correo y fecha de creación.

Paso 3: Gestionar usuarios

Puedes deshabilitar, eliminar o restablecer contraseñas desde esta interfaz.

En Firestore:

Los datos de tareas y categorías están en Firestore → Database → Data. Puedes ver, editar y eliminar documentos directamente desde la consola.

3.2 Restablecer contraseña de un usuario

Paso 1: Firebase Console

Authentication → Users → busca el usuario → clic en los tres puntos → "Send password reset email".

3.3 Eliminar un usuario

 *Eliminar un usuario desde Firebase Authentication NO elimina sus datos en Firestore. Debes eliminar manualmente la colección del usuario.*

Paso 1: Eliminar de Authentication

Authentication → Users → selecciona el usuario → Delete.

Paso 2: Eliminar datos de Firestore

Firestore → Data → colección usuarios → busca el documento del usuario → Delete.

4. Administración de la Base de Datos

4.1 Estructura de la base de datos MySQL

Tabla	Registros típicos	Descripción
usuarios	Uno por cuenta	Datos del usuario: nombre, email, hash contraseña.
tareas	Muchos por usuario	Tareas con título, prioridad, fecha, estado, pomodoros.
categorias	6+ por usuario	Categorías con nombre, color e iconos.
alertas	Una por tarea	Recordatorios programados con hora de disparo.

4.2 Respaldo de la base de datos

Se recomienda hacer un respaldo semanal de la base de datos MySQL.

Exportar desde la terminal:

```
mysqldump -u root -p taskmaster_db > backup_taskmaster_YYYY-MM-DD.sql
```

Restaurar un respaldo:

```
mysql -u root -p taskmaster_db < backup_taskmaster_YYYY-MM-DD.sql
```

4.3 Respaldo de Firestore

Desde Firebase Console → Firestore → Exports → Exportar a Google Cloud Storage. Se recomienda programar exportaciones automáticas semanales desde la consola de GCP.

5. Deploy en Producción — Hostinger

5.1 Pasos de deploy

Paso 1: Contratar plan VPS

Hostinger VPS KVM 1 o superior. Seleccionar servidor en USA. Anotar IP del servidor.

Paso 2: Crear BD en Hostinger

hPanel → MySQL → crear taskmaster_db → importar taskmaster_db.sql.

Paso 3: Actualizar .env

Cambiar DB_HOST, DB_USER, DB_PASSWORD con los datos de Hostinger.

Paso 4: Subir archivos

hPanel → Administrador de archivos → public_html → subir y extraer ZIP del proyecto.

Paso 5: Activar Node.js

hPanel → Node.js → Crear app → server.js → npm install automático.

Paso 6: Configurar dominio

hPanel → Dominios → apuntar al servidor. HTTPS se activa automáticamente.

5.2 Variables de entorno en producción

En Hostinger, las variables de entorno se configuran en hPanel → Node.js → tu aplicación → Environment Variables. Agrega las mismas variables del .env sin el archivo físico.

 En producción NUNCA usar root como usuario de MySQL. Crea un usuario específico con solo los permisos necesarios.

6. Monitoreo y Mantenimiento

6.1 Verificar el estado del servidor

```
# Ver si el servidor está corriendo:
curl https://tudominio.com/api/materias
# Respuesta esperada: {"success":true,"materias":[...]}
```

6.2 Ver logs de errores

```
# En Hostinger:
hPanel → Node.js → tu aplicación → View Error Logs
```

```
# En local:
npm start
# Los logs aparecen en la terminal
```

6.3 Actualizar el sistema

Paso 1: Hacer pull del repositorio

git pull origin avance-eduardo

Paso 2: Reinstalar dependencias

npm install

Paso 3: Reiniciar el servidor

En Hostinger: hPanel → Node.js → Restart. En local: Ctrl+C y npm start.

6.4 Tareas de mantenimiento periódico

Frecuencia	Tarea	Cómo hacerlo
Semanal	Respaldo de BD	mysqldump o exportar desde phpMyAdmin
Semanal	Respaldo de Firestore	Firebase Console → Exports
Mensual	Actualizar dependencias	npm audit y npm update en el servidor
Mensual	Revisar logs de errores	hPanel → Node.js → Error Logs
Trimestral	Renovar credenciales OAuth	Google Console → rotar client secrets

7. Seguridad del Sistema

7.1 Medidas de seguridad implementadas

- Contraseñas hashadas con bcryptjs factor de costo 10 — nunca se guarda texto plano
- CORS configurado para permitir solo dominios autorizados
- Variables de entorno para datos sensibles — nunca en el código fuente
- Firebase Admin SDK con credenciales de servicio en archivo JSON privado
- HTTPS obligatorio en producción (SSL de Hostinger)
- Parámetros preparados en consultas SQL — prevención de inyección SQL

7.2 Credenciales y accesos

⚠ Cambiar las credenciales por defecto inmediatamente después de instalar en producción.

Credencial	Dónde cambiarla
Contraseña MySQL root	phpMyAdmin → Cuentas de usuario → Cambiar contraseña
Firebase Admin SDK	Firebase Console → Configuración → Cuentas de servicio → Generar nueva clave

Credencial	Dónde cambiarla
Google OAuth secrets	Google Cloud Console → APIs → Credenciales → rotar secret
Variables .env	Servidor → hPanel → Node.js → Environment Variables



INSTITUTO SUPERIOR TECNOLÓGICO RUMIÑAHUI

MANUAL DE DESARROLLADOR

TaskMaster Web — Sistema de Gestión de Productividad Personal

Versión 1.0 — Agosto 2026

Eduardo Antonio Hurtado Quintero

Ing: Llumiquinga Espinoza Fabricio Jose

Sangolquí – Ecuador, Agosto 2026

1. Arquitectura del Sistema

TaskMaster Web implementa una arquitectura de tres capas con separación estricta de responsabilidades. El frontend usa el patrón MVVM en JavaScript puro. El backend tiene dos servidores independientes: el servidor principal en Node.js/Express y el servidor de integraciones. La base de datos primaria es Firebase Firestore con MySQL como respaldo relacional.

1.1 Diagrama de capas

FRONTEND	BACKEND PRINCIPAL	INTEGRACIONES
HTML5 SPA (index.html)	Node.js 18 + Express	Node.js + Express
CSS3 con variables	Firebase Admin SDK	Google OAuth 2.0
JavaScript MVVM puro	bcryptjs autenticación	Microsoft OAuth 2.0
Service Worker (PWA)	googleapis npm	Brevo (email)
Firebase JS SDK	PHP 8 + mysqli	ICS Builder

1.2 Patrón MVVM — Flujo de datos

El frontend implementa Model-View-ViewModel. El flujo es unidireccional: View → ViewModel → Model → API. Las actualizaciones regresan: API → Model → ViewModel (notifica) → View (se redibuja).

Capa	Archivos	Responsabilidad
Model	TaskModel.js CategoriaModel.js	Fetch a API REST. Fallback a localStorage. Sin lógica de negocio.
ViewModel	TaskViewModel.js	Estado global. Patrón Observer (subscribe/notify). Validación. Coordina Model y Views.
View	HomeView.js ModalView.js SemanaView.js PomodoroView.js AuthView.js	Solo renderiza DOM. Escucha eventos del ViewModel. No hace fetch directo.

2. Estructura de Archivos

2.1 Raíz del proyecto

Archivo/Carpeta	Descripción
server.js	Servidor principal Node.js. API REST con Firebase Admin. Endpoints de tareas, auth, materias.
package.json	Dependencias: express, bcryptjs, cors, dotenv, firebase-admin, googleapis.
index.html	SPA completa con todas las vistas y modales. Scripts cargados al final del body.
sw.js	Service Worker para PWA. Cachea el app shell para modo offline.
manifest.json	Configuración PWA: nombre, íconos, colores, modo standalone.
lib/firestore.js	Instancia singleton de Firebase Admin Firestore.
.env	Variables de entorno. NO incluir en Git.
integraciones/	Servidor secundario para OAuth Google/Microsoft, Calendar, Drive, Teams.
api/	Endpoints PHP legacy: tareas, login, registro, categorias, stats.
models/	Modelos PHP: Tarea.php, Categoria.php, usuarios.php.
controllers/	Controladores PHP: TareasController, CategoriasController, UsuariosController.
database/	taskmaster_db.sql (schema), taskmaster.sql (con datos), datospruebas.sql.
docs/	API.POSTMAN.json, DER.png, CATEGORY_ICONS.md, Manual.pdf.

3. Backend — Server.js Principal

3.1 Inicialización y Firebase

```
import { db } from './lib/firestore.js';
db.collection('_healthcheck').limit(1).get()
  .then(() => console.log('✅ Firebase Firestore conectado'))
  .catch(err => console.error('❌ Error:', err.message));
```

→ **db es la instancia de Firestore obtenida desde lib/firestore.js**

→ **El healthcheck verifica la conexión al iniciar el servidor**

3.2 Función auxiliar siguienteId()

```
async function siguienteId(coleccion) {
  const snap = await db.collection(coleccion).get();
  let max = 0;
  snap.forEach(doc => {
    const id = Number(doc.data().id || 0);
    if (id > max) max = id;
  });
  return max + 1;
}
```

→ **Firestore no tiene AUTOINCREMENT — esta función simula el incremento de id**

→ **Lee todos los documentos de la colección y devuelve el id más alto + 1**

3.3 Endpoints implementados

Método	Ruta	Colección Firestore	Descripción
GET	/api/materias	materias	Lista categorías. Si vacía, inicializa las 6 por defecto.
GET	/api/tareas	tareas	Lista tareas. Soporta filtro por usuario_id.
POST	/api/tareas	tareas	Crea tarea con id autogenerado.
PUT	/api/tareas/:id	tareas	Actualiza campos específicos por id de documento.
DELETE	/api/tareas/:id	tareas	Elimina documento de Firestore.
POST	/api/auth	usuarios (Firestore)	Login y registro. bcrypt.hash/compare.

4. Frontend — Módulos JavaScript

4.1 TaskModel.js — Capa de datos

Responsable de toda comunicación con la API REST. Implementa fallback automático a localStorage cuando la API no responde.

```
async getAll() {
  try {
    const res = await fetch(this.API_TAREAS);
    const data = await res.json();
    if (data.success) {
      this._tareas = data.tareas;
      localStorage.setItem('tm_tareas',
        JSON.stringify(this._tareas));
      return this._tareas;
    }
  } catch (e) { /* sin conexión */ }
  return JSON.parse(localStorage.getItem('tm_tareas') || '[]');
}
```

→ **Patrón de red primero, caché como respaldo (Network First)**

4.2 TaskViewModel.js — Estado global

Implementa el patrón Observer. Las vistas se suscriben y se notifican automáticamente.

```
subscribe(fn) { this._listeners.push(fn); }
_notify(evento, payload = {}) {
  this._listeners.forEach(fn => fn(evento, {...this._state,
    ...payload}));
}
```

→ **subscribe()** registra una función observadora

→ **_notify()** dispara el evento a todos los observadores registrados

4.3 Service Worker — PWA

sw.js implementa el patrón Cache First para el app shell y Network First para la API.

```
const CACHE_NAME = 'taskmaster-shell-v2';
// install: cachea todos los archivos del APP_SHELL
// fetch: sirve desde cache primero, red como respaldo
// Las rutas /api/ siempre van a la red (sin cachear)
```

→ **Actualizar CACHE_NAME al hacer cambios para forzar actualización del cache**

 Para probar PWA en local: Chrome DevTools → Application → Service Workers

5. Servidor de Integraciones

5.1 Estructura

El servidor de integraciones corre en un puerto separado y gestiona OAuth 2.0 con Google y Microsoft, y las rutas de Calendar, Drive, Teams, Classroom y OneDrive.

Ruta/Módulo	Funcionalidad
routes/oauthGoogle.js	Flujo OAuth 2.0 con Google. Genera token de acceso y refresh token.
routes/oauthMicrosoft.js	Flujo OAuth 2.0 con Microsoft. Acceso a Teams y OneDrive.
routes/googleCalendar.js	Lee eventos de Google Calendar y los convierte en tareas.
routes/googleDrive.js	Lista y adjunta archivos de Google Drive a tareas.
routes/classroom.js	Importa actividades de Google Classroom como tareas.
routes/teams.js	Envía notificaciones de tareas urgentes a canales de Teams.
routes/oneDrive.js	Lista y adjunta archivos de OneDrive a tareas.
routes/email.js	Envío de emails de recordatorio usando Brevo (Sendinblue).
routes/ics.js	Genera archivos .ics para importar eventos en cualquier calendario.
lib/tokenStore.js	Almacena y recupera tokens OAuth de forma segura.

5.2 Configurar Google OAuth

- Ve a console.cloud.google.com → Crear proyecto → APIs → Credenciales
- Crear credencial OAuth 2.0 → Aplicación web
- URI de redireccionamiento: <http://localhost:3001/auth/google/callback>
- Copiar Client ID y Client Secret al .env de integraciones

6. Base de Datos — Firebase Firestore y MySQL

6.1 Colecciones en Firestore

Colección	Campos principales	Notas
tareas	id, titulo, prioridad, fecha_limite, completada, usuario_id, pomodoros_est, pomodoros_real	El campo id es gestionado manualmente con siguienteId()
materias	id, nombre, color, bg	Se inicializan 6 categorías por defecto si la colección está vacía
usuarios	id, nombre, email, password_hash, fecha_creacion	password_hash generado con bcryptjs factor 10
_healthcheck	— (solo para verificar conexión)	Colección temporal para verificar Firestore al iniciar

6.2 Tablas MySQL (respaldo relacional)

MySQL se usa como sistema de respaldo y para los endpoints PHP legacy. Las tablas son: usuarios, tareas, categorias y alertas con relaciones mediante FK y ON DELETE CASCADE.

6.3 Reglas de Firestore

Las reglas de Firestore se configuran en Firebase Console → Firestore → Rules. En producción se deben restringir para que cada usuario solo acceda a sus propios datos:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /tareas/{tareaId} {
      allow read, write: if request.auth != null
        && request.auth.uid == resource.data.usuario_id;
    }
  }
}
```


7. Referencia de la API REST

7.1 Autenticación

POST /api/auth — Registro

```
Body: { nombre, email, password }  
Respuesta OK: { success: true, user: { id, nombre, email } }  
Error: { success: false, error: 'El correo ya está registrado.' }
```

POST /api/auth — Login

```
Body: { email, password, action: 'login' }  
Respuesta OK: { success: true, user: { id, nombre, email } }  
Error: { success: false, error: 'Correo o contraseña incorrectos.' }
```

7.2 Tareas

GET /api/tareas

```
Query params: ?usuario_id=123 (opcional)  
Respuesta: { success: true, tareas: [ {...}, {...} ] }
```

POST /api/tareas

```
Body: { titulo, prioridad, fecha_limite, materia_id, usuario_id,  
        descripcion, pomodoros_est, min_anticipacion }  
Respuesta: { success: true, tarea: { id, titulo, ... } }
```

PUT /api/tareas/:id

```
Body: { completada: 1 } // o cualquier campo a actualizar  
Respuesta: { success: true }
```

DELETE /api/tareas/:id

```
Respuesta: { success: true }
```

 El archivo docs/API.POSTMAN.json contiene la colección completa de Postman para probar todos los endpoints.

8. Control de Versiones — Git

8.1 Estructura de ramas

Rama	Propósito
main	Rama principal del repositorio. Código estable.
avance-eduardo	Rama activa de desarrollo. Contiene los 4 avances académicos y correcciones.

8.2 Historial de commits

```
f870a2a ← Correcciones CSS, UI y funcionalidad de botones (HEAD)
13bbb86 ← Avance 4 - 100%: Autenticación, categorías y rediseño
5ed3da4 ← Avance 3 - 75%: Dashboard, filtros y UI
0b0c2ab ← Avance 2 - 50%: JS MVVM y API REST PHP
b73ae29 ← Avance 1 - 25%: Estructura HTML, CSS y base de datos
```

8.3 Flujo de trabajo para nuevas funcionalidades

```
git checkout avance-eduardo
git pull origin avance-eduardo
# hacer cambios...
git add .
git commit -m 'feat: descripción del cambio'
git push origin avance-eduardo
```

9. Convenciones y Estándares de Código

9.1 JavaScript — Frontend

- Clases en PascalCase: TaskModel, HomeView, TaskViewModel
- Métodos privados con guión bajo: _notify(), _renderCard()
- Variables con camelCase: tareaActual, idUsuario
- Constantes con MAYÚSCULAS: API_TAREAS, CACHE_NAME
- Async/await en lugar de callbacks o .then()
- Optional chaining (?.) para acceso seguro a propiedades

9.2 Node.js — Backend

- ES Modules (import/export) en lugar de CommonJS (require)
- Async/await con try/catch para manejo de errores
- Respuestas siempre con { success: true/false, ... }
- Variables de entorno con process.env.NOMBRE || "default"

9.3 CSS

- Variables CSS en :root para colores, espaciados y tipografía
- Clases con guión: .task-card, .btn-primary, .modal-overlay
- Modificadores con doble guión: .task-card--done, .chip--active
- Mobile first en responsive.css

9.4 Mensajes de commit (Conventional Commits)

Prefijo	Cuándo usarlo
feat:	Nueva funcionalidad. Ej: feat: agregar filtro por fecha
fix:	Corrección de bug. Ej: fix: modal no cerraba con Escape
docs:	Cambios en documentación. Ej: docs: actualizar README
style:	Cambios de CSS sin afectar funcionalidad.
refactor:	Reorganización del código sin cambiar comportamiento.
chore:	Tareas de mantenimiento. Ej: chore: actualizar dependencias



INSTITUTO SUPERIOR TECNOLÓGICO RUMIÑAHUI

MANUAL DE USUARIO

TaskMaster Web — Sistema de Gestión de Productividad Personal

Versión 1.0 — Agosto 2026

Eduardo Antonio Hurtado Quintero

Ing: Llumiquinga Espinoza Fabricio Jose

Sangolquí – Ecuador, Agosto 2026

1. Introducción

TaskMaster Web es un sistema de gestión de productividad personal accesible desde cualquier navegador web. Permite organizar tareas por categorías, establecer prioridades y fechas límite, configurar recordatorios personalizados y usar la técnica Pomodoro para maximizar el enfoque. El sistema también se puede instalar como aplicación en el dispositivo (PWA) y funciona sin conexión a internet.

Este manual está dirigido a los usuarios finales del sistema: cualquier persona que quiera organizar sus actividades personales, laborales, académicas o del hogar desde un navegador web o desde la aplicación instalada en su celular o computadora.

1.1 Requisitos para usar TaskMaster

Requisito	Detalle
Navegador web	Google Chrome 90+, Firefox 88+, Edge 90+ o Safari 14+
Conexión a internet	Requerida para sincronizar. Sin internet funciona en modo offline.
Cuenta de usuario	Registro con nombre, correo y contraseña. También con Google.
Dispositivo	Computadora, tablet o celular con cualquier sistema operativo.

2. Registro e Inicio de Sesión

2.1 Crear una cuenta nueva

Para usar TaskMaster por primera vez debes crear una cuenta personal. Sigue estos pasos:

Paso 1: Abrir la aplicación

Abre tu navegador y escribe la dirección del sitio web de TaskMaster. Verás la pantalla de bienvenida con las opciones de Login y Registro.

Paso 2: Seleccionar Registro

Haz clic en el botón "Crear cuenta" o "Registrarse". Se abrirá el formulario de registro.

Paso 3: Completar el formulario

Ingresa tu nombre completo, tu correo electrónico y una contraseña de al menos 8 caracteres. La contraseña se guarda de forma segura — nadie puede verla.

Paso 4: Confirmar registro

Haz clic en "Crear cuenta". Si todo está correcto, ingresarás automáticamente a la aplicación.

 Consejo: Puedes también iniciar sesión con tu cuenta de Google haciendo clic en el botón "Continuar con Google". No necesitas crear una contraseña nueva.

 Importante: Usa un correo real al que tengas acceso. Lo necesitarás si olvidas tu contraseña.

2.2 Iniciar sesión en una cuenta existente

Paso 1: Abrir la pantalla de login

Abre la aplicación. Si no hay sesión activa, verás el formulario de inicio de sesión.

Paso 2: Ingresar credenciales

Escribe tu correo y contraseña y haz clic en "Iniciar sesión". La sesión se guarda automáticamente para que no tengas que volver a ingresar cada vez.

2.3 Cerrar sesión

Para cerrar tu sesión ve al menú de perfil en la esquina superior derecha y haz clic en "Cerrar sesión". Esto elimina la sesión de tu navegador.

3. Pantalla Principal — Mis Tareas

Al iniciar sesión llegas a la pantalla principal donde están todas tus tareas. Tiene tres secciones principales: la barra de navegación, el área de filtros y la lista de tareas.

3.1 Elementos de la pantalla principal

Elemento	Para qué sirve
Header / Barra superior	Muestra el logo, el nombre de la app y el botón de tu perfil.
Botón + (FAB)	Botón flotante para crear una nueva tarea rápidamente.
Chips de filtro	Filtran las tareas por categoría: Todas, Trabajo, Estudio, Personal, etc.
Tarjeta de tarea	Muestra el título, categoría, prioridad, fecha límite y la barra de urgencia.
Barra de urgencia	Barra de color que indica qué tan cerca está la fecha límite.
Navegación inferior	Botones para ir a Mis Tareas, Mi Semana o Enfoque Pomodoro.

4. Crear y Gestionar Tareas

4.1 Crear una nueva tarea

Hay dos formas de crear una tarea:

- Clic en el botón + (esquina inferior derecha)
- Clic en el botón "Nueva tarea" en el header

Se abre un formulario con los siguientes campos:

Campo	Obligatorio	Descripción
Título	☒ Sí	Nombre de la tarea. Máximo 200 caracteres.
Descripción	No	Detalles adicionales de la tarea.
Categoría	No	Estudio, Trabajo, Personal, Salud, Finanzas, General.
Prioridad	No	Alta (rojo), Media (naranja), Baja (verde). Por defecto: Media.
Fecha límite	☒ Sí	Fecha y hora en que debe estar lista la tarea.
Recordatorio	No	Minutos antes de la fecha para recibir una notificación.
Pomodoros estimados	No	Cuántas sesiones de 25 min crees que tomará completarla.

💡 Consejo: Puedes escribir en el título frases como "Reunión mañana urgente" y el sistema detecta la fecha y prioridad automáticamente.

4.2 Editar una tarea

Haz clic sobre la tarjeta de la tarea que quieres editar. Se abre el formulario con todos los datos actuales. Modifica lo que necesites y haz clic en "Guardar".

4.3 Marcar una tarea como completada

Haz clic en el checkbox o botón de completar en la tarjeta de la tarea. La tarjeta se oscurece y el texto aparece tachado. Puedes desmarcarla de la misma forma.

4.4 Eliminar una tarea

Abre la tarea haciendo clic en ella. En el formulario de detalle aparece el botón "Eliminar". El sistema pide confirmación antes de borrar definitivamente.

 **Importante:** Una tarea eliminada no se puede recuperar. Asegúrate antes de confirmar.

5.1 Filtrar por categoría

En la parte superior de la lista de tareas hay chips de colores con el nombre de cada categoría. Haz clic en uno para ver solo las tareas de esa categoría. Haz clic en "Todas" para volver a ver todo.

5.2 Filtrar por estado

Puedes filtrar entre "Pendientes" (tareas sin completar) y "Completadas" usando los chips correspondientes

5.3 Colores de urgencia

Color	Significado	Cuándo aparece
 Verde	Tiempo suficiente	Más de 3 días hasta la fecha límite
 Amarillo	Atención necesaria	Entre 1 y 3 días hasta la fecha límite
 Rojo	Urgente	Menos de 24 horas o ya vencida

6. Vista Semanal — Mi Semana

La sección "Mi Semana" muestra un gráfico de barras con la distribución de tareas por día de la semana. Te ayuda a ver si tienes días sobrecargados y a redistribuir tu trabajo.

6.1 Cómo usar la vista semanal

Paso 1: Ir a Mi Semana

Toca el ícono de calendario en la barra de navegación inferior.

Paso 2: Ver el gráfico

Cada barra representa un día. La altura indica cuántas tareas tienes ese día. Si una barra está en rojo, ese día tiene más de 5 tareas (sobrecarga).

Paso 3: Navegar semanas

Usa los botones ← y → para ver semanas anteriores o futuras.

💡 Consejo: Si ves que un día está en rojo, considera mover algunas tareas a días con menos carga para tener un balance mejor.

7. Temporizador Pomodoro

La técnica Pomodoro divide el trabajo en sesiones de 25 minutos seguidas de un descanso de 5 minutos. TaskMaster tiene un temporizador integrado que puedes iniciar desde cualquier tarea.

7.1 Iniciar una sesión Pomodoro

Paso 1: Ir a Enfoque

Toca el ícono de tomate 🍅 en la barra de navegación inferior.

Paso 2: Seleccionar la tarea

Elige la tarea en la que vas a trabajar.

Paso 3: Iniciar el timer

Haz clic en el botón de reproducción ►. El anillo circular comienza a vaciarse.

Paso 4: Trabajar sin interrupciones

Concéntrate en la tarea durante 25 minutos. Cuando termine, recibirás una notificación.

Paso 5: Tomar un descanso

Descansa 5 minutos. Luego puedes iniciar otra sesión.

💡 Consejo: El número de sesiones completadas se guarda en la tarea. Así puedes ver cuánto tiempo dedicaste a cada actividad.

7.2 Controles del Pomodoro

Botón	Acción
► Iniciar	Comienza el temporizador de 25 minutos.
⏸ Pausar	Pausa el temporizador. El tiempo queda guardado.
■ Reiniciar	Vuelve el timer a 25:00 sin contar la sesión.

8. Notificaciones y Recordatorios

8.1 Activar notificaciones

La primera vez que abres TaskMaster, el navegador pregunta si deseas recibir notificaciones. Haz clic en "Permitir" para activarlas. Estas notificaciones aparecen en tu sistema operativo aunque tengas el navegador en otra pestaña o minimizado.

8.2 Tipos de notificaciones

- Recordatorio de tarea: aparece según los minutos de anticipación que configuraste al crear la tarea.

- Fin de Pomodoro: aparece cuando se completan los 25 minutos de una sesión de enfoque.
- Tarea próxima a vencer: aparece automáticamente cuando una tarea tiene menos de 1 hora para vencer.

 **Importante:** Si rechazaste las notificaciones al inicio, puedes activarlas desde la configuración de tu navegador → permisos del sitio.

9. Modo Offline e Instalación como App

9.1 Modo offline

TaskMaster funciona aunque no tengas internet. Si pierdes la conexión, la app usa automáticamente los datos guardados localmente en tu navegador. Cuando recuperas la conexión, se sincronizan los cambios automáticamente.

9.2 Instalar TaskMaster como app

Puedes instalar TaskMaster en tu celular o computadora como si fuera una app nativa. No necesitas ir a la Play Store ni al App Store.

En Android (Chrome):

Paso 1: Abrir el menú

Toca los tres puntos : en la esquina superior derecha de Chrome.

Paso 2: Instalar app

Toca "Instalar aplicación" o "Agregar a pantalla de inicio".

Paso 3: Confirmar

Toca "Instalar". TaskMaster aparece como ícono en tu pantalla de inicio.

En computadora (Chrome o Edge):

Paso 1: Ver el ícono

En la barra de direcciones aparece un ícono de instalación .

Paso 2: Instalar

Haz clic en el ícono y luego en "Instalar". TaskMaster se abre como ventana independiente.

10. Integraciones con Otras Herramientas

TaskMaster se conecta con las herramientas que ya usas para que no tengas que ingresar datos dos veces.

Integración	Para qué sirve
Google Calendar	Importa tus eventos como tareas automáticamente. Las tareas con fecha también aparecen en tu calendario.
Gmail	Envía recordatorios por correo electrónico cuando una tarea está próxima a vencer.
Google Drive	Adjunta archivos de Drive directamente a una tarea.
Google Classroom	Importa actividades y tareas de tus clases como tareas en TaskMaster.
Microsoft Teams	Recibe notificaciones de tareas urgentes directamente en tus canales.
OneDrive	Adjunta archivos de OneDrive a tus tareas.

 Consejo: Para activar una integración ve al menú de configuración y selecciona "Integraciones". Haz clic en la integración que deseas y sigue los pasos de autorización.

11. Preguntas Frecuentes

¿Puedo usar TaskMaster sin crear una cuenta?

No. Necesitas una cuenta para que tus tareas queden guardadas en la nube y puedas acceder desde cualquier dispositivo.

¿Qué pasa si olvido mi contraseña?

En la pantalla de login toca "¿Olvidaste tu contraseña?" e ingresa tu correo. Recibirás un enlace para crear una nueva contraseña.

¿Mis tareas se borran si cierro el navegador?

No. Las tareas se guardan en Firebase Firestore (nube) y en el localStorage de tu navegador. Están disponibles siempre que inicies sesión.

¿Puedo usar TaskMaster en varios dispositivos?

Sí. Inicia sesión con tu cuenta en cualquier dispositivo y verás todas tus tareas sincronizadas en tiempo real gracias a Firebase Firestore.

¿Las notificaciones funcionan en el celular?

Sí, si instalas TaskMaster como PWA en tu celular. Las notificaciones del navegador móvil funcionan igual que las notificaciones de apps nativas.